Learning Objectives

- Deepen your understanding of singly linked lists and their implementation.
- Use `readonly` to express and enforce that data must not be modified.
- Recognize trade-offs between simple, efficient, and maintainable implementations.
- Design and implement a simple symbol table for global and local variables.
- Separate the interface of a data structure from its implementation.

Step 1 — Revisiting Singly Linked Lists

In Worksheet 7, we made our first experiments with singly linked lists and dynamically allocated memory. Initially, we wrote all the code directly in main. We then started moving useful pieces of code into separate functions — for example, the code for printing a list into `printList`.

We will now finish this process. At the end of this step, you will have the complete list implementation that was shown in the Chill Out section of Worksheet 7.

Start with the following program `list.abc`:

```
@ <stdio.hdr>
@ <stdlib.hdr>

struct Node
{
    val: int;
    next: -> Node;
};

// To fix!
fn prependToList(list: ->Node, val: int)
{
    local n: ->Node = malloc(sizeof(*n));
    assert(n);
    n->val = val;
    n->next = list;
    list = n;
}

fn printList(list: ->Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
    }
}

fn printListRecursive(list: ->Node)
{
    if (list) {
        printf("val = %d\n", list->val);
        printListRecursive(list->next);
    }
}

fn freeList(list: ->Node)
{
    // TODO
}

fn main()
{
    local list: ->Node = nullptr;

    for (local i: int = 1; i <= 42; ++i) {
        prependToList(list, i);
    }

    printf("list:\n");
    printList(list);

    freeList(list);
}
```

Tasks:

1. Compile and run the program. Something is wrong: although `prependToList` is called `42` times, the list printed by `printList` is empty. Figure out why `prependToList` does not work as intended. In particular, think about what happens to the parameter `list` when the function is called.

© Michael C. Lehn | CC BY-SA 4.0

2. Change `prependToList` so that it can actually modify the variable `list` in main.

   You will have to change both the parameter type of `prependToList` and the way the function is called.

   Spoiler: On worksheet 7 you can find a possible solution in the Chillout section.
3. Add a function

   ```
   fn printListReverse(list: ->Node)
   ```

   that prints the elements in reverse order without modifying the list. Call `printListReverse` from `main` and verify that the elements are printed in reverse order.

   Hint: Compare printListRecursive with what you need. At what point should the current value be printed relative to the recursive call?
4. Complete `freeList`. It has to free all dynamically allocated nodes. Be careful: once a node has been passed to `free`, you must no longer access it.

   Spoiler: On worksheet 7 you can find a possible solution in the Chillout section.

Step 2 — Motivation for `readonly`

When we call a function that prints a list, we normally expect the function not to modify the list. But suppose that, due to an unfortunate copy-and-paste mistake, `printList` was changed as follows:

```
fn printList(list: -> Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
        list->val *= 2;
    }
}
```

After printing the value of a node, the function now doubles it. Make this change in your program and test it. As expected, you will observe that the list is modified every time `printList` is called.

Now change the function as follows:

```
fn printList(list: -> readonly Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
        list->val = 42;
    }
}
```

The parameter `list` is still a pointer to a list node. However, we now use the keyword `readonly`. In this declaration, it specifies that **the object the pointer points to must not be modified through this pointer**.

Compile the program. The compiler now reports an error:

```
MCL:session16 lehn$ abc list_1.abc
list_1.abc:26.14-26.17: error: : assignment of read-only variable 'list->val'

        list->val = 42;
                  ^^
list_1.abc:26.18-26.20: error: operator can not be applied to operand 'list->val' of type
'const int' and  operand '42' of type 'i32'
```

Why is getting an error a good thing? Errors are of course not good in themselves, but an error at compile time is much better than discovering an unintended modification at runtime.
By using `readonly`, we have documented — both for the programmer and for the compiler — that this function must not modify the contents of a list node. The compiler detects that the assignment `list->val = 42` breaks this promise.

We now have two choices: either the function really is supposed to modify the nodes, in which case we remove readonly, or the modification was unintended, in which case we fix the bug.

© Michael C. Lehn | CC BY-SA 4.0

Step 3 — General Use of `readonly`

The use of `readonly` is not restricted to pointer types. It can be used with any variable type.

Create a file `foo.abc` with the following example:

```
@ <stdio.hdr>

fn main()
{
    local a: readonly int;

    a = 42;
    printf("a = %d\n", a);
}
```

Here, a is declared as `readonly`, which means that its value must not be changed after initialization.

Compile the program and inspect the compiler error.

For a local variable, declaring it as `readonly` without initializing it is therefore not particularly useful: we cannot assign a value to it afterwards, and an uninitialized local variable has an undefined value. We should usually initialize it directly:

```
local a: readonly int = 42;
```

But what if the value used for initialization depends on a condition? Consider the following example. The user enters a value for `n`. If n is greater than `10`, we want a to have the value `42`; otherwise, it should have the value `13`:

```
 1 @ <stdio.hdr>
 2
 3 fn main()
 4 {
 5     local n: int;
 6     scanf("%d", &n);
 7
 8     local a: readonly int;
 9     if (n > 10) {
10         a = 42;
11     } else {
12         a = 13;
13     }
14
15     printf("a = %d\n", a);
16 }
```

This cannot be compiled because both assignments attempt to modify a after its declaration. Instead, we can determine the value as part of the initialization:

```
local a: readonly int = n > 10 then 42 else 13;
```

The expression on the right-hand side is called a *conditional expression*. If `n > 10` is `true`, its value is `42`; otherwise, its value is `13`.

Change the program accordingly and test it again.

Takeaway: Even if you are not particularly fond of conditional expressions, they become very useful once `readonly` — or `const` in C and C++ — enters the picture.

Step 4 — Back to the List

Task: After this short excursion, return to `list.abc`. Change all functions that only print the list so that they take a pointer of type

```
-> readonly Node
```

as their parameter. Compile and test your program again.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Searching for a Node

Task: Write a function `isInList` that checks whether a given value occurs in the list. The function shall return `true` if the value is found and `false` otherwise.

Choose an appropriate parameter type for list. Remember that `isInList` only inspects the list and must not modify it.

In `main`, use the function as follows:

```
printf("5 is %s list\n", isInList(list, 5) ? "in" : "not in");
printf("42 is %s list\n", isInList(list, 42) ? "in" : "not in");
```

Compile and test your program.

Step 6 — Appending Nodes the Ugly and Inefficient Way (Don't Do That at Home!)

So far, we can only insert new nodes at the beginning of the list. For a singly linked list, this is both the simplest and the most efficient way to add a node.

However, we can also append a new node at the end of the list. We want to implement a function `appendToList` that can be used in `main` just like `prependToList`:

```
83      local list: -> Node = nullptr;
84
85      for (local i: int = 1; i <= 10; ++i) {
86          prependToList(&list, i);
87          appendToList(&list, i + 10);
88      }
```

The implementation of `appendToList` is considerably more complicated than that of function `prependToList`.

Task: Implement

```
fn appendToList(list: ->-> Node, val: int)
```

Proceed step by step:

1. Allocate a new node, store `val` in it, and make sure that the new node has no successor.
2. First consider the case of an **empty list**. What must `*list` point to after the operation?
3. Now consider a **non-empty list**. Follow the `next` pointers until you reach the last node and append the new node there.
4. Call `appendToList` from `main` as shown above and use `printList` to verify that your implementation works for both an empty and a non-empty list.

The Bad News

Although this works, it is not a particularly good way to implement this operation.

Compare `appendToList` with `prependToList`. Appending requires us to distinguish between an empty and a non-empty list. For a non-empty list, we also have to traverse the entire list before we can append the new node.

There is an even more important difference: the running time of `prependToList` does not depend on the length of the list. For `appendToList`, however, the longer the list becomes, the longer it takes to find its end.

We could change our data structure to make appending simpler and more efficient. For now, however, keep this deliberately ugly implementation — it is good practice for working with pointers and linked lists.

© Michael C. Lehn | CC BY-SA 4.0

Step 6 — Practical Application — A Simple Symbol Table

Linked lists in their pure form are among the least efficient data structures for searching: in the worst case, we have to traverse the entire list to find an element. The access time therefore depends on the length of the list. On the other hand, linked lists are very flexible: we do not have to know in advance how many elements we are going to store.
Arrays are often much more efficient when it comes to access times. The software developer Scott Meyers once summarized this observation as

“I don't know [data structure], but I know an array will beat it.”

This statement is less exaggerated than it may sound. For many practical problems, arrays are indeed faster than considerably more sophisticated data structures because of their simple memory layout and good cache utilization. We will discuss this in more detail in HPC1.

In this worksheet, we will not consider data structures such as hash maps, which could provide a good compromise here. Instead, we will use a linked list for a simple but useful application in HPC0: **a symbol table**.

A symbol table is a data structure used by a compiler to keep track of information about identifiers. In our case, we want to store global and local variables. A compiler does not only need to know whether an identifier exists. During code generation, it also needs to know what the identifier refers to and, for a local variable, where it can be found.

Our symbol table shall support the following operations:

- `symtabAddGlobal((String)"foo")` tries to add a new global variable named `foo`. If no global variable with this name exists yet, the function adds it and returns `true`. Otherwise, it returns `false`.
- `symtabAddLocal((String)"foo", 8)` tries to add a new local variable named `foo` with offset `8`. If no local variable with this name exists yet, the function adds it and returns `true`. Otherwise, it returns `false`. A global variable with the same name does not prevent us from adding the local variable.
- `symtabIsDefined((String)"foo")` returns `true` if `foo` is currently defined as either a local or a global variable, and `false` otherwise.
- `symtabGetInfo((String)"foo", &info)` looks up `foo`. If the identifier is found, the function returns `true` and stores information about it in `info`. If both a local and a global variable with this name exist, the **local variable takes precedence**. The member `info.isGlobal` tells us whether the identifier refers to a global or local variable. For a local variable, `info.offset` contains its offset. If the identifier is not defined, the function returns `false`.
- `symtabPrintGlobals()` and `symtabPrintLocals()` print all global and local variables. For global variables, “printing” means generating assembler code for the BSS segment.
- `symtabFreeLocals()` forgets all local variables. A compiler needs this operation when it finishes code generation for one function and starts processing another.

The interface is described more formally in `symtab.hdr`:

```
@ "string.hdr"

struct SymtabInfo
{
    isGlobal: bool;
    offset: int;
};

extern fn symtabGetInfo(ident: String, info: -> SymtabInfo): bool;
extern fn symtabIsDefined(ident: String): bool;

extern fn symtabAddGlobal(ident: String): bool;
extern fn symtabAddLocal(ident: String, offset: int): bool;
extern fn symtabFreeLocals();

extern fn symtabPrintGlobals();
extern fn symtabPrintLocals();
```

The included file `string.hdr` contains just a single line:

```
type String: array[42] of char;
```

A variable of type `String` is therefore simply an array of exactly 42 characters, including the terminating null byte.

© Michael C. Lehn | CC BY-SA 4.0

In short: Strings are a surprisingly complex topic in C-like languages.
A little less short: This is certainly not the best possible representation of strings. How to handle strings “better” is a topic for a later session. What “better” means depends on what we want to optimize for: a simple implementation, low memory consumption, fast access, flexible length, or safe memory management.
C, C++, and therefore ABC do not prescribe one particular solution. There is no single “correct” representation — only different design decisions with different advantages and disadvantages.
We will now implement the symbol table. Before we start, however, we need a small test program. The current version of `xtest_symtab.abc` is still far from complete:

```
@ <stdio.hdr>

@ "symtab.hdr"

fn main()
{
    // add some global and local variables
    assert(symtabAddGlobal((String)"foo"));
    symtabAddLocal((String)"bar", -8);
    printf("GLOBAL VARIABLES:\n");
    symtabPrintGlobals();
    printf("LOCAL VARIABLES:\n");
    symtabPrintLocals();

    assert(symtabIsDefined((String)"foo"));
    assert(!symtabAddGlobal((String)"foo"));

    local info: SymtabInfo;
    symtabGetInfo((String)"foo", &info);
    assert(info.isGlobal);

    symtabAddLocal((String)"bar", -8);
    symtabGetInfo((String)"bar", &info);
    assert(!info.isGlobal);
    assert(info.offset == -8);

    printf("GLOBAL VARIABLES:\n");
    symtabPrintGlobals();
    printf("LOCAL VARIABLES:\n");
    symtabPrintLocals();
    symtabFreeLocals();
}
```

Task: Compiling the Test Program

Create the files `string.hdr`, `symtab.hdr`, and `xtest_symtab.abc` shown above.
So far, our programs consisted of a single source file. This time, things will be different: the implementation of the symbol table will live in a separate file `symtab.abc`.
But let us first see what already works without it. Compile `xtest_symtab.abc` without creating an executable:

```
abc -c xtest_symtab.abc
```

This should succeed and produce the object file `xtest_symtab.o`.

Think about it: How can the compiler translate calls such as

```
symtabAddGlobal((String)"foo")
```

although we have not implemented `symtabAddGlobal` yet?
The declarations in `symtab.hdr` tell the compiler everything it needs to know to compile the call: the name of the function, its parameter types, and its return type. The actual implementation is not needed yet.

Now try to create an executable:

```
abc xtest_symtab.abc -o xtest_symtab
```

This time, the build must fail. Look at the error messages. You should see that functions such as `symtabAddGlobal`, `symtabIsDefined`, and `symtabGetInfo` are missing.

Note that this is a linker error, and not a compiler error. The compiler was able to translate `xtest_symtab.abc`, but when creating the executable, the linker could not find the implementations of the functions declared in `symtab.hdr`.

© Michael C. Lehn | CC BY-SA 4.0

Step 7 — Implementing the Symbol Table

We now provide the missing implementations in a second source file, `symtab.abc`:

```
@ <stdlib.hdr>
@ <stdio.hdr>
@ <string.hdr>

@ "symtab.hdr"

struct Node {
    ident: String;
    offset: int;
    next: -> Node;
};

global globalVar, localVar: -> Node;

fn symtabGetInfo(ident: String, info: -> SymtabInfo): bool
{
    assert(0 && "TODO");
    return false;
}

fn symtabIsDefined(ident: String): bool
{
    assert(0 && "TODO");
    return false;
}

fn symtabAddGlobal(ident: String): bool
{
    assert(0 && "TODO");
    return false;
}

fn symtabAddLocal(ident: String, offset: int): bool
{
    assert(0 && "TODO");
    return false;
}

fn symtabFreeLocals()
{
    assert(0 && "TODO");
}

fn symtabPrintGlobals()
{
    assert(0 && "TODO");
}

fn symtabPrintLocals()
{
    assert(0 && "TODO");
}
```

For now, provide stubs for all functions declared in `symtab.hdr`. The stubs do not have to implement the symbol table yet; they only have to provide definitions that can be linked.

We now have two translation units: `xtest_symtab.abc` and `symtab.abc`. Compile and link them together:

```
abc xtest_symtab.abc symtab.abc -o xtest_symtab
```

Run the resulting program:

```
./xtest_symtab
```

The linker should now be satisfied because the functions declared in `symtab.hdr` have definitions in `symtab.abc`. The program itself will, of course, still fail one of its assertions because our functions are only stubs. Keep the test program unchanged. As you implement the functions step by step, more and more of its tests will succeed.

© Michael C. Lehn | CC BY-SA 4.0

Task:

1. Start with `symtabAddGlobal` and `symtabAddLocal`. For the first version, always insert a new node at the beginning of the corresponding list, even if an entry with the same identifier already exists. For now, both functions therefore always succeed and return `true`.
2. Next, implement `symtabPrintGlobals` and `symtabPrintLocals`. Use them to check that nodes are actually inserted into the correct lists.
   If `globalVar` contains nodes for the identifiers `x` and `y`, `symtabPrintGlobals` shall produce output of the following form:

   ```
       .bss
       .align 8
   x:
       .quad
       .align 8
   y:
       .quad
   ```

   The output resembles the assembler code generated by a compiler for uninitialized global variables. Assume `localVar` contains nodes for `x` and `y` with offsets `-8` and `24`, respectively, then `symtabPrintLocals` shall produce:

   ```
   # x with offset -8
   # y with offset 24
   ```

   The `#` makes each line an assembler comment, so we can later include this output directly in generated assembly code. **Note:** Since new nodes are inserted at the beginning of a list, the entries may be printed in reverse order. The order does not matter.
3. We now have to prevent duplicate definitions. Implement a helper function

   ```
   fn isInList(list: ->readonly Node, ident: String): ->readonly Node
   ```

   that searches for `ident` in the given list and returns a pointer to the corresponding node if found, or `nullptr` otherwise.
   This is very similar to the `isInList` function you implemented in Step 5. There is, however, one important difference: `ident` is now a string. Strings cannot be compared for equality using `==`. In `<string.hdr>` — the header from the standard library, not our own `"string.hdr"` — the function `strcmp` is declared. It compares two strings:

   ```
   strcmp(s1, s2)
   ```

   returns `0` if the strings are equal, a negative value if `s1` comes before `s2` lexicographically, and a positive value if `s1` comes after `s2`. For our purpose, we therefore only need:

   ```
   strcmp(s1, s2) == 0
   ```

   Remark: In our program, `s1` and `s2` can be of type `String`, i.e. arrays of 42 characters. For `strcmp(s1, s2)`, the ABC compiler implicitly turns the arguments into pointers to the first characters of the arrays.
4. Use `isInList` to implement `symtabIsDefined`. An identifier is defined if it occurs in either `localVar` or `globalVar`.
5. Use `isInList` in `symtabAddGlobal` and `symtabAddLocal`. Before inserting a new node, check whether the identifier already exists in the corresponding list. If it does, return `false` without inserting a node.
6. Now implement `symtabGetInfo`. This function has to do more than just determine whether an identifier exists: it has to find the corresponding entry and return information about it.
   Search the local variables first. If the identifier is found there, set

   ```
   info->isGlobal = false;
   info->offset = ...;
   ```

   and return `true`. Only if no local variable with this name exists, search the global variables. If it is found there, set

   ```
   info->isGlobal = true;
   ```

   and return `true`. If the identifier is not found at all, return `false`. Why is it important to search `localVar` before `globalVar`?
7. Finally, implement `symtabFreeLocals`. Free all nodes in `localVar` and make the list empty again. **Extend your test program** so that after calling `symtabFreeLocals` a previously defined local variable is no longer found, the same local name can be added again, and if a local variable was hiding a global variable with the same name, `symtabGetInfo` finds the global variable again.

© Michael C. Lehn | CC BY-SA 4.0